

Plan: Gömb outer-shell throughput (2026-05-12)

Scope. Vectorize `scatter_mean` and batch the M parallel Clifford-FIR banks in `OuterFIRShell` (`signedkan_wip/src/hymeko_gomb/shells.py`) so forward matches the prior numerics (bit-exact is not required; floating-point associativity should match PyTorch defaults for the same operation order where possible).

Affected files. `signedkan_wip/src/hymeko_gomb/shells.py`, `signedkan_wip/tests/test_hymeko_gomb.p`

CORE.YAML. None.

Interface changes. None public beyond `scatter_mean` accepting batched input (`B`, `M_c`, `d`) in addition to (`M_c`, `d`) (documented in rustdoc-style comment in the module).

Test strategy. Unit tests: parity test for `scatter_mean` vs a reference loop implementation (2D and 3D); existing Gömb tests unchanged expectations.

Performance budget. Wall time for `OuterFIRShell.forward` should decrease on GPU for $M \geq 4$ at fixed (N, M_c, k, d) ; peak RSS unchanged order-of-magnitude (no materialized $O(M \cdot N \cdot M_c)$ temporaries beyond the batched path already implied by the algorithm).

Rollback. Revert `shells.py` to sequential bank loop + corner loop in `scatter_mean`.

Risks. Stacking `nn.Linear` weights each forward adds small CPU overhead; acceptable versus eliminated Python loops over M and k .